Airline-Passenger-Segmentation

October 31, 2026

1 Airline Passenger Segmentation

This analysis explores passenger groups based on age and arrival delay using K-means clustering.

It complements the airline review analysis in this repository by examining a separate dataset. The two datasets are analysed independently, and satisfaction rates are not directly compared with recommendation rates.

1.1 Analysis Questions

- What passenger groups emerge from age and arrival delay?
- How do the groups differ in size, age and delay?
- Do satisfaction rates differ across these groups?

1.2 Data Source and Overview

Dataset name:Airline Customer Satisfaction Dataset
Link:https://www.kaggle.com/datasets/mehwishzehra/airline-customer-satisfaction-dataset

The dataset contains 129,880 records and 22 columns. Arrival Delay in Minutes has 393 missing values.

Age and Arrival Delay in Minutes are used for clustering. Satisfaction is reserved for comparing the resulting groups and is not used to form the clusters.

This analysis was adapted from a data science coursework project.

[2]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

b_df = pd.read_csv("Invistico_Airline.csv")

print(b_df.shape)
print(b_df.isna().sum().sort_values(ascending=False).head(10))
print(b_df.dtypes)
```

```
(129880, 22)
Arrival Delay in Minutes      393
Customer Type                   0
```

```
Departure Delay in Minutes      0
Online boarding                 0
Cleanliness                     0
Checkin service                 0
Baggage handling                0
Leg room service                0
On-board service                0
Ease of Online booking          0
dtype: int64
satisfaction                           object
Customer Type                          object
Age                                     int64
Type of Travel                         object
Class                                  object
Flight Distance                         int64
Seat comfort                            int64
Departure/Arrival time convenient       int64
Food and drink                          int64
Gate location                           int64
Inflight wifi service                   int64
Inflight entertainment                  int64
Online support                          int64
Ease of Online booking                  int64
On-board service                        int64
Leg room service                        int64
Baggage handling                        int64
Checkin service                         int64
Cleanliness                             int64
Online boarding                         int64
Departure Delay in Minutes              int64
Arrival Delay in Minutes              float64
dtype: object
```

1.3 B.2 — Missing Data Treatment

[36]:
```
# selected_features= ["Age", "Arrival Delay in Minutes"]

# print("Missing values before treatment:")
# print(b_df[selected_features].isna().sum())

# feature_medians= b_df[seleted_features].median()

# b_df[seleted_features]= b_df[seleted_features].fillna(
#     feature_medians
#     )

# print("\nMissing values after treatment:")
```

```
# print(b_df[selected_features].isna().sum())
```

[37]:
```
selected_features = ["Age", "Arrival Delay in Minutes"]

print(selected_features)
print(type(selected_features))
```

```
['Age', 'Arrival Delay in Minutes']
<class 'list'>
```

[38]:
```
print(type(b_df))
print(b_df.shape)
print(type(selected_features))
print(selected_features)
```

```
<class 'pandas.core.frame.DataFrame'>
(129880, 22)
<class 'list'>
['Age', 'Arrival Delay in Minutes']
```

[9]:
```
age_median = b_df["Age"].median()
delay_median = b_df["Arrival Delay in Minutes"].median()

b_df["Age"] = b_df["Age"].fillna(age_median)
b_df["Arrival Delay in Minutes"] = (
    b_df["Arrival Delay in Minutes"].fillna(delay_median)
)

print("Missing values after treatment:")
print(b_df[["Age", "Arrival Delay in Minutes"]].isna().sum())
```

```
Missing values after treatment:
Age                         0
Arrival Delay in Minutes    0
dtype: int64
```

There are 393 missing values within Arrival Delay in Minutes column originally, while the Age columns had no missing values.I fill missing values using the median of each column because the median is less affected by extreme values. After treatment, both columns had zero missing values.

At the end of the statement, i need to clearify that it keep going error when i run the entire code, so then i handled the two selected features separately so that each column was filled with its own median and each step could be checked clearly.

1.4 B.3 — Feature Scaling and K-means Clustering

[10]:
```
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

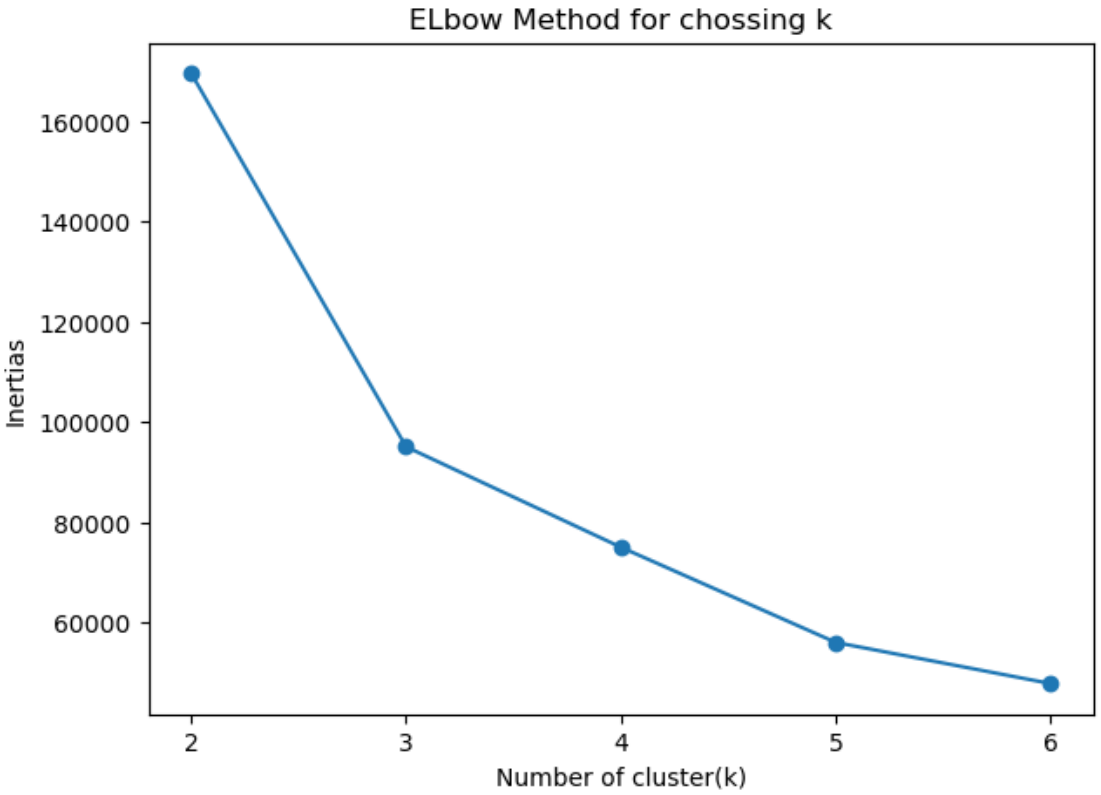
cluster_features = ["Age","Arrival Delay in Minutes"]
```

```
b_scaler = StandardScaler()
b_scaled = b_scaler.fit_transform (b_df[cluster_features])

inertias=[]

for k in range(2,7):
    model = KMeans(
        n_clusters= k,
        random_state=42,
        n_init=10
        )
    model.fit(b_scaled)
    inertias.append(model.inertia_)

plt.figure(figsize=(7,5))
plt.plot(range(2, 7), inertias, marker="o")
plt.xlabel("Number of cluster(k)")
plt.ylabel("Inertias")
plt.title("ELbow Method for chossing k")
plt.xticks(range(2,7))
plt.show()
```



According to the elbow plot, the inertia decreases sharply from k=2 to 3. After k= 3, the decrease in inertia is leveling off. Therefore , i chose k=3 because it provides a balance between reducing within-cluster variation and avoiding too many cluster.

[11]:
```
chosen_k =3

kmeans= KMeans(
    n_clusters=chosen_k,
    random_state=42,
    n_init=10
)

b_df["cluster"]= kmeans.fit_predict(b_scaled)

print("Chosen k:",chosen_k)
print(b_df["cluster"].value_counts().sort_index())
```

```
Chosen k: 3
cluster
0    66431
1     5872
2    57577
Name: count, dtype: int64
```

[6]:
```
print(b_df["satisfaction"].value_counts(dropna=False))
```

```
satisfaction
satisfied       71087
dissatisfied    58793
Name: count, dtype: int64
```

[15]:
```
# Mark satisfied passengers as True
b_df["is_satisfied"] = b_df["satisfaction"].eq("satisfied")

# Calculate a profile for each cluster
cluster_summary = b_df.groupby("cluster").agg(
    Passenger_Count=("Age", "size"),
    Average_Age=("Age", "mean"),
    Average_Arrival_Delay_Minutes=("Arrival Delay in Minutes", "mean"),
    Satisfaction_Rate_Pct=("is_satisfied", "mean")
)

# Convert satisfaction rates to percentages
cluster_summary["Satisfaction_Rate_Pct"] *= 100

display(cluster_summary.round(2).T)
```

cluster	0	1	2
Passenger_Count	66431.00	5872.00	57577.00
Average_Age	51.20	38.96	25.89
Average_Arrival_Delay_Minutes	8.42	152.12	8.72
Satisfaction_Rate_Pct	63.12	42.12	46.35

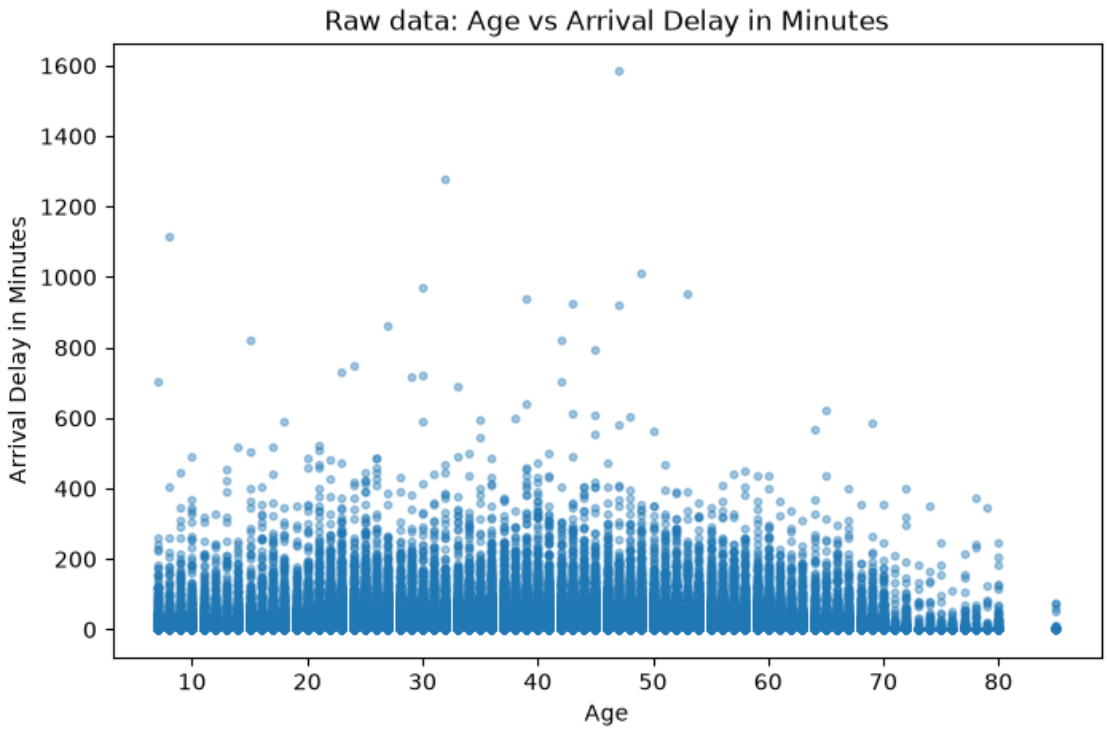
Cluster 1 has the longest average arrival delay, at 152.12 minutes, and the lowest satisfaction rate, at 42.12%. Clusters 0 and 2 have similar average delays of around 8–9 minutes, but their satisfaction rates are 63.12% and 46.35%, respectively—a difference of 16.77 percentage points. This suggests that delay alone does not explain the differences in satisfaction. Further analysis could examine travel class, travel purpose and service ratings. These results show associations and do not prove that age or arrival delay causes changes in satisfaction.

1.5 Cluster Visualisation and Profiles

[42]:
```
# plot the raw data
plt.figure(figsize=(8,5))

plt.scatter(
    b_df["Age"],
    b_df["Arrival Delay in Minutes"],
    alpha=0.4,
    s=10
)

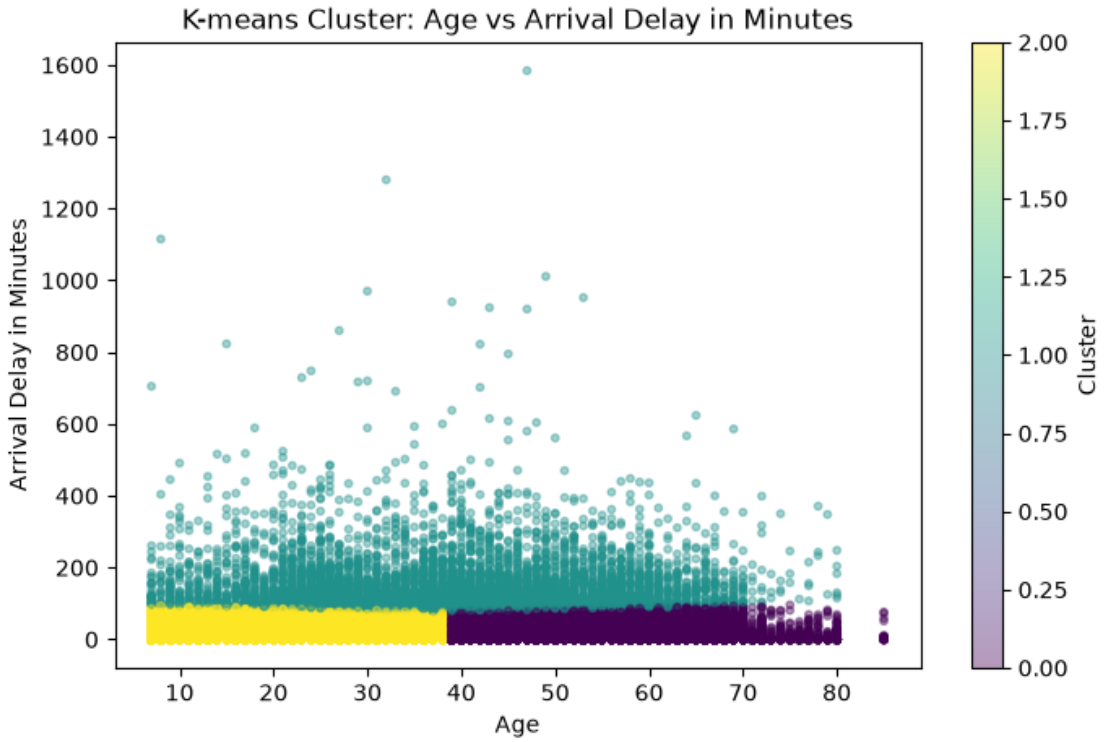
plt.xlabel("Age")
plt.ylabel("Arrival Delay in Minutes")
plt.title("Raw data: Age vs Arrival Delay in Minutes")
plt.show()
```



[46]:
```
# plot the cluster results
plt.figure(figsize=(8,5))

plt.scatter(
    b_df["Age"],
    b_df["Arrival Delay in Minutes"],
    c=b_df["cluster"],
    cmap= "viridis",
    alpha=0.4,
    s=10
)

plt.xlabel("Age")
plt.ylabel("Arrival Delay in Minutes")
plt.title("K-means Cluster: Age vs Arrival Delay in Minutes")
plt.colorbar(label = "Cluster")
plt.show()
```



[47]:
```
centres_scaled = kmeans.cluster_centers_

centres_original = b_scaler.inverse_transform(centres_scaled)

centres_df = pd.DataFrame(
    centres_original,
    columns=cluster_features
)

centres_df.index.name = "Cluster"
display(centres_df)
```

Cluster	Age	Arrival Delay in Minutes
0	51.199154	8.415017
1	38.967456	152.154882
2	25.894269	8.719868

The cluster centres represent the average age and arrival delay for each cluster in the original units.

1.6 Method Rationale

I chose this dataset because it contains meaningful numerical information, such as passenger age , arrival delay time, and satisfaction scores for seat comfort and other services. the Arrival Delay in

Minutes column contains 393 missing values ,so the dataset is suitable for demonstrating missing-data treatment. i used k-means to group passenger with similar ages and arrival delay times. The algorithm repestedly assigns passenger to the nearest cluster centere and updates the centre of each cluster. I standardised the two features before clustering because they have different numerical ranges. *

1.7 Findings and Limitations

Before clustering, I expected passengers with similar arrival delays to be grouped together. The results showed one group with high arrival delays and two groups with low arrival delays. The two low-delay groups differed mainly in age, with average ages of approximately 51 and 26 years. This shows that the clustering reflects differences in both age and arrival delay. However, the groups overlap in the scatter plot, and the results depend on the selected features and the choice of three clusters. The differences in satisfaction rates do not establish a causal relationship.